Top-75 Discriminative Terms per Amazon Review Category via MapReduce on Hadoop

Group X — Data-Intensive Computing (194.048), TU Wien
Member 1, Member 2, Member 3, Member 4, Member 5

April 28, 2026

1 Introduction

Identifying the most discriminative vocabulary of a document collection is a recurring task in text mining: it powers feature selection for classification, domain-specific dictionary construction, and exploratory analysis of large review corpora. In this assignment we apply the chi-square (χ^2) test of independence to the Amazon Reviews dataset to extract, for each of the 22 product categories, the 75 unigram terms most strongly associated with that category. The full corpus comprises about 78.8 million review records distributed across an HDFS file of approximately 58 GB, which makes the problem genuinely data-intensive and motivates a MapReduce solution.

The pipeline is implemented with the `mrjob` Python framework and executed on the TU Wien LBD Hadoop cluster (Hadoop 3.3.6, YARN). All correctness and performance numbers reported here come from a single end-to-end run on the full dataset that completed in 19 min 24 s of wall-clock time.

2 Problem Overview

Input and output

The input is a line-delimited JSON file in which every record contains at least a `category` field and a `reviewText` field. The required output is a single text file with 23 lines:

- 22 category lines, sorted alphabetically by category name. Each line starts with the category name followed by 75 `word:chi2` tokens, sorted by descending χ^2 score.
- 1 final line with the merged dictionary: the alphabetically sorted union of all words selected as top-75 in any category.

Chi-square statistic

For each (term t, category c) pair we build a 2×2 contingency table over the documents of the corpus:

	in category c	not in category c
contains t	A	C
does not contain t	B	D

The values are derived from three aggregates: cat_total_c (documents in c), $token_total_t$ (documents containing t across all categories), and N (total documents):

$$B = cat_total_c - A, \quad C = token_total_t - A, \quad D = N - cat_total_c - token_total_t + A.$$

The score is then

$$\chi^2(t,c) \;=\; \frac{N\,(A\,D - B\,C)^2}{token_total_t\,(N - token_total_t)\,cat_total_c\,(N - cat_total_c)}. \tag{1}$$

Document frequency, not term frequency

A subtle but critical detail: A is a *document frequency*, i.e. the number of documents in c that contain t at least once, not the number of times t occurs. An implementation that emits one count per token *occurrence* would inflate A and can even violate $A \leq cat_total_c$, breaking the contingency table. Our mapper therefore deduplicates tokens within each document before emission (see Section 3.2).

Dataset characteristics (from cluster counters)

The full run reports $N = 78\,828\,876$ reviews and 22 categories. After preprocessing, the post-stopword vocabulary is approximately 6.2 million distinct words; the number of distinct (c, t) pairs to be scored is 12,189,683. The fact that the vocabulary grows sublinearly with collection size (Heaps' law) is what makes a single-reducer pass over the post-aggregated stream tractable at this scale; we return to this point in the discussion.

3 Methodology

3.1 Pipeline overview

Figure 1 shows the conceptual pipeline. The implementation realises it as four `MRSteps` linked by `mrjob`. Phase 3 of the figure corresponds to two physical MRSteps in the code: one to gather token totals across categories (MRStep 2) and one to compute and rank χ^2 scores (MRStep 3); Phase 4 of the figure corresponds to MRStep 4 (final assembly). All four steps run on the same cluster job submission and share `mrjob`'s temporary HDFS directory.

3.2 MRStep 1 — Preprocessing and document-frequency counting

Mapper. For every input record, the JSON line is parsed with `ujson` (falling back to the standard `json` module if `ujson` is not installed on the cluster image). The `reviewText` field is lower-cased and split on a single regular expression that treats all required punctuation, whitespace, tabs and digits as delimiters in one pass. Each token is filtered against the stopword set (loaded once in `mapper_init`) and the minimum-length rule ($|w| > 1$). The surviving tokens are inserted into a per-document *set*, and only *after* the document is fully tokenised does the mapper emit one record per distinct token:

```
seen = set()
for word in self.splitting_on.split(review_text):
    if word and word not in self.stopwords and len(word) > 1:
        seen.add(word)
for word in seen:
    yield (category, word), 1
yield (category, '__TOTAL__'), 1
```

The deduplication ensures that downstream A values are document frequencies. The extra `(category, '__TOTAL__')` record carries the per-category document count needed to derive cat_total and N in MRStep 2.

Combiner. An in-mapper combiner sums counts locally before they are shuffled to the reducer. On the full run the combiner reduces 2.34 billion intermediate records to 144 million, a shrink factor of 16× that is essential to keep the shuffle volume manageable (Table 1).

Reducer. A single reducer sums counts globally. Its output is 12.19 million $((c, w) \rightarrow count)$ records and 22 $((c, \texttt{__TOTAL__}) \rightarrow cat_total)$ records.

3.3 MRStep 2 — Rekey and token-total computation

To compute $token_total_t = \sum_c A_{c,t}$, all per-category counts of the same token must arrive at the same reducer task. The mapper therefore rekeys the stream by token:

```
def mapper_2(self, key, count):
    cat, word = key
    if word == '__TOTAL__':
        yield '__TOTAL__', (cat, count)
    else:
        yield word, (cat, count)
```

The reducer accumulates the 22 `__TOTAL__` records into a dictionary, derives N, and then for each token computes *token_total* on the fly and emits one record per category in which the token appears, carrying the four values needed for χ^2:

```
yield (cat, word), [A, token_total, cat_total, self.N]
```

This step is configured with `jobconf={'mapreduce.job.reduces': '1'}`. The reason is that the `__TOTAL__` sentinel must be visible to every word reducer call, and the cleanest way to guarantee this in `mrjob` is to funnel every key through one task; the JSON-encoded key `"__TOTAL__"` sorts before any lower-case word, so the dictionary is fully populated by the time word records arrive. The trade-off is sequential reduce work; we discuss its actual cost in Section 3.6 and a parallelisable alternative in Section 4.

3.4 MRStep 3 — Chi-square, top-75, partial dictionary

The mapper computes Equation 1 directly and rekeys by category. Records with a degenerate denominator (e.g. a token appearing in every document, or an empty category) are dropped.

Bounded top-k with `heapq`. The combiner and reducer both use `heapq.nsmallest(75, values, key=lambda x: (-x[0], x[1]))` to pick the local top 75 by $-\chi^2$ with alphabetic tie-break. This is linear in the number of values and uses $\mathcal{O}(75)$ extra memory, unlike a full `sorted()`. The combiner alone shrinks this step's shuffle from 12.18 million records to 18 150 (a factor of 670×, see Table 1); without the combiner the single Step-3 reducer would be the dominant bottleneck.

Merged dictionary. Every reducer call adds the selected words to a shared `self.dict_words` set; `reducer_final` emits the partial dictionary under the sentinel key `~DICT~`.

3.5 MRStep 4 — Final assembly

The mapper funnels every Step-3 record under one synthetic key (`__OUT__`), which forces all of them into a single reducer call (the data volume is 23 records, so this is essentially free). The reducer separates category lines from `~DICT~` partials, sorts the categories alphabetically, unions the partials into the final dictionary line, and emits everything via `RawValueProtocol` so that the `mrjob` stdout is already in the required `output.txt` format — no post-processing script is needed.

3.6 Cluster results

The job was submitted to the LBD cluster with

```
time python main.py \
  --hadoop-streaming-jar /usr/lib/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
  -r hadoop --upload-file stopwords.txt \
  hdfs:///dic_shared/amazon-reviews/full/reviewscombined.json \
  > output.txt 2> log.txt
```

and finished in **19 min 24 s** wall-clock time (real 19m24.763s). Table 1 summarises the per-step Hadoop counters extracted from `log.txt`.

Splits and parallelism. MRStep 1 was split into 435 map tasks (launched 446, 11 killed); MRSteps 2–4 used 3, 9 and 2 map tasks respectively. The total accumulated map CPU time was

Step	Map in	Map out	Combiner shrink	Reduce in/out	Reduce time
1	78 828 876	2 214 174 909	2.34B→144M (16×)	21.4 M / 12.2 M	597 s
2	12 189 683	12 189 683	—	12.2 M / 12.2 M	172 s
3	12 175 531	12 175 531	12.2M→18k (670×)	18 150 / 23	5 s
4	23	23	—	23 / 23	5 s

Table 1: Per-step Hadoop counters for the full-dataset run. Combiner shrink is reported only where a combiner is configured. Reduce time is the total time spent in reduce tasks for the step (each step launched exactly one reducer).

about 23.6 hours, distributed across the cluster's available containers; the wall-clock map phase of MRStep 1 fitted within the run because YARN scheduled roughly two hundred concurrent map containers.

Data reduction. The 58 GB input is reduced to 395 MB after MRStep 1, a 147× reduction driven by the combiner. By the end of MRStep 3 the relevant data is only 42 kB (Step 3 `Bytes Written`); the final `output.txt` is 41.97 kB.

4 Conclusions

The pipeline produces the required output and processes the full 58 GB corpus in under 20 minutes wall-clock time. Three design choices contributed most to this result:

1. **Document-frequency deduplication in the mapper**, which keeps the chi-square contingency table well-formed.
2. **Combiners on Steps 1 and 3**, both of which reduce shuffle volume by more than an order of magnitude (16× and 670×). In particular, the bounded top-k combiner in Step 3 turns a potentially expensive global sort into a linear-time selection with constant per-mapper memory.
3. **Output-side simplifications**: `RawValueProtocol` on the last step removes the need for a separate formatting script, and the single-key funnel in Step 4 keeps the final assembly deterministic across multiple reducers.

The principal limitation is the *single-reducer constraint of MRStep 2*, which is forced by the `__TOTAL__` broadcast pattern. At our data scale this cost only ∼3 minutes (172 s reduce time) thanks to Heaps' law: the post-aggregation stream is bounded by the vocabulary size, not by the raw review count. For substantially larger corpora the constraint would become the bottleneck, and a cleaner refactor is the *distributed-cache / side-input pattern*: a tiny preceding job materialises the per-category totals into a small file, the main job loads it via `-upload-file` in `mapper_init`, and the `__TOTAL__` plumbing disappears entirely. MRStep 2 then becomes embarrassingly parallel across an arbitrary number of reducers. We did not implement this refactor here because the current solution comfortably meets the runtime target.

A further point worth noting is that the actual wall-clock cost of the job is split roughly evenly between MRStep 1 (raw I/O and tokenisation on 58 GB) and the sequential reduce phases of MRSteps 1 and 2; subsequent steps are computationally trivial. This is consistent with the general observation that, at this scale, MapReduce jobs are dominated by the cost of the first heavy-aggregation pass, and that careful combiner design pays back on every byte of input.

Processing Pipeline Draft.pdf

Figure 1: Conceptual pipeline. Phase 0 loads stopwords once per mapper task (`mapper_init`). Phase 1 does the heavy lifting on the raw 58 GB input. Phase 2 is `mrjob`'s internal shuffle. Phase 3 computes the contingency-table aggregates, evaluates χ^2, and selects the top 75 per category. Phase 4 emits the 22 category lines and the merged dictionary in the required `output.txt` format.